

Kalman Filter

Milestone 4 Report

RISC-V Vector (RVV) Assembly Implementation

Computer Architecture and Assembly Language

Team Thunder

Ali Hamza (30618)

Huzaifa Iqbal (31563)

Muhammad Sufyan Sohail (31567)

Muhammad Usman Hussain (28992)

Date: 24th May 2026

Abstract

This report presents the vectorised implementation of the Linear Kalman Filter (LKF) and Extended Kalman Filter (EKF) using the RISC-V Vector extension (RVV 1.0), building on the scalar assembly code from Milestone 3. The main mathematical operations such as matrix multiplication, addition, subtraction, transpose, and symmetrisation, were rewritten using vector instructions, which allow the processor to work on multiple numbers at once instead of one at a time. This reduces the number of instructions needed by around 7 to 8 times for most operations. The results were tested against the Milestone 3 scalar output over 3,040 frames and 276 state variables. Both filters produced outputs that were completely identical to the scalar reference, with a maximum error of zero, which is far below the allowed tolerance of $\varepsilon = 10^{-9}$. All 23 joints and all state components (position, velocity, acceleration, and jerk) passed verification for both the LKF and EKF.

Contents

1	Introduction and Purpose	3
2	Background: RISC-V Vector Extension	3
2.1	How RVV Works	3
2.2	Key Instructions Used	3
2.3	Strip-Mining	4
3	Vectorised LKF Implementation (<code>lkf_vector.s</code>)	4
3.1	<code>asm_mat_mul</code> : Vectorised Matrix Multiplication	4
3.2	<code>asm_mat_add</code> : Vectorised Matrix Addition	6
3.3	<code>asm_mat_sub</code> : Vectorised Matrix Subtraction	7
3.4	<code>asm_mat_transpose</code> : Vectorised Transpose	7
3.5	<code>asm_symmetrize</code> : Vectorised Symmetrisation	8
3.6	Scalar Fallback Functions	9
4	Vectorised EKF Implementation (<code>ekf_vector.s</code>)	10
4.1	<code>asm_vec_zero_matrix</code>	11
4.2	<code>asm_vec_mat_copy</code>	11
4.3	<code>asm_vec_negate</code>	12
4.4	<code>asm_vec_add_identity</code>	13
4.5	Architectural Justification for Remaining Scalar Functions	13
5	Memory Management and Alignment	14
5.1	The 8-Byte Alignment Requirement	15
5.2	Handling Strided Memory Accesses	15
5.3	Going Faster: Cache Line Alignment (64 Byte)	15
6	Numerical Verification	16
6.1	Overall Results	16
6.2	Table 1: Average Absolute Error per Joint	17
6.3	Table 2: Average Absolute Error per State Component	17
6.4	Table 3: Global Maximum and Minimum Absolute Error	18
6.5	Table 4: Direct M3 vs M4 Comparison (Pelvis, Frame 1)	18
7	Performance Analysis	19
7.1	Experimental Setup	19
7.2	Instruction Count Reduction	19
7.3	Measured Wall-Clock Speedup	20
7.4	Why the Speedup Is Lower Than the Instruction Count Reduction	20
7.5	Visual Performance Analysis and Graph Observations	22
7.5.1	Detailed Breakdown of Graph Observations	22
7.5.2	Connecting the Three Graphs	23
7.6	Projected Performance on Physical Hardware	24
7.7	Conclusion of Performance Analysis	24
8	Build and Run Instructions	25

9	Conclusion	25
9.1	Closing Remarks	26
9.2	Code Availability and Repository Link	27

1 Introduction and Purpose

The goal of Milestone 4 is to make the Kalman filter programs from Milestone 3 run faster. We do this using the RISC-V Vector extension (RVV 1.0), which allows the processor to work on many numbers at the same time instead of one number at a time. This is very useful for matrix operations, because the same calculation is repeated for every element in the matrix.

The system we are working with tracks human body movement across 23 body joints. Each joint has 12 state variables including position, velocity, acceleration, and jerk in the x , y , and z directions and so giving a total of $N = 12 \times 23 = 276$ state variables. Each joint also has 3 measurement sensors, meaning $M = 3 \times 23 = 69$ measurements in total. The most expensive part of both filters is matrix multiplication on these large matrices, which makes them the best place to apply vectorisation.

The two main files produced in this milestone are `lkf_vector.s`, which contains the vectorised versions of the core math functions used by both filters, and `ekf_vector.s`, which contains extra helper functions written specifically for the Extended Kalman Filter using RVV instructions.

2 Background: RISC-V Vector Extension

2.1 How RVV Works

RVV introduces 32 vector registers (`v0–v31`). Unlike scalar registers, each vector register holds multiple elements. The number of elements processed per instruction, called the *vector length* (VL), depends on three things:

1. **VLEN**: the hardware's physical vector register width in bits (set to 128 bits in our QEMU simulation via `vlen=128`).
2. **Element width (SEW)**: we use `e64` (64-bit doubles), so each element takes 8 bytes.
3. **LMUL**: the register grouping multiplier. We use `m4`, meaning four vector registers are merged into one group. This quadruples the effective vector length.

With $VLEN = 128$ bits, $SEW = 64$ bits, and $LMUL = m4$:

$$VL = \frac{VLEN}{SEW} \times LMUL = \frac{128}{64} \times 4 = 8 \text{ doubles per instruction}$$

This means one `vfmac.vf` instruction performs 8 fused multiply/add operations simultaneously, compared to a scalar loop that would need 8 separate `fmul/fadd` pairs.

2.2 Key Instructions Used

Table 1 summarises the RVV instructions most heavily used in this milestone.

Table 1: Key RVV instructions used in the vectorised implementation.

Instruction	What it does
<code>vsetvli t, n, e64, m4, ta, ma</code>	Set VL to <code>min(n, hardware_max)</code> and store it in <code>t</code> . This handles loop tails automatically.
<code>vle64.v vd, (ptr)</code>	Load VL consecutive doubles from memory into register group <code>vd</code> .
<code>vse64.v vs, (ptr)</code>	Store VL doubles from register group <code>vs</code> to memory.
<code>vlse64.v vd, (ptr), stride</code>	Load VL doubles spaced <code>stride</code> bytes apart (strided load).
<code>vsse64.v vs, (ptr), stride</code>	Store VL doubles with <code>stride</code> bytes between them (strided store).
<code>vfmacc.vf vd, fs, vs</code>	<code>vd[i] += fs * vs[i]</code> for all i in $[0, \text{VL})$. The scalar <code>fs</code> is broadcast to every lane.
<code>vfadd.vv vd, vs1, vs2</code>	Element-wise addition: <code>vd[i] = vs1[i] + vs2[i]</code> .
<code>vfsb.vv vd, vs1, vs2</code>	Element-wise subtraction.
<code>vfmul.vf vd, vs, fs</code>	Multiply every element of <code>vs</code> by scalar <code>fs</code> .
<code>vfmv.v.f vd, fs</code>	Broadcast scalar <code>fs</code> to all elements of <code>vd</code> .

2.3 Strip-Mining

Because matrices are typically larger than what fits in one vector register group, we use a technique called *strip-mining*. The loop runs in chunks of size VL, calling `vsetvli` at the start of every iteration. When the remaining element count is less than the hardware maximum, the processor automatically reduces VL so the last partial chunk is handled correctly and so no explicit tail handling code is needed.

3 Vectorised LKF Implementation (`lkf_vector.s`)

The file `lkf_vector.s` contains vector-accelerated versions of the five core math functions that were previously written as scalar code in `kalman_common.s`. The important thing to note is that all function names and argument lists are kept exactly the same as before. This means the C driver files and the filter logic files (`lkf_asm.s` and `ekf_asm.s`) do not need any modifications. The only difference at build time is which object file gets linked in, `lkf_vector.o` instead of `kalman_common.o`.

3.1 `asm_mat_mul`: Vectorised Matrix Multiplication

What it computes

$$C_{M \times N} = A_{M \times K} \times B_{K \times N}$$

This function multiplies matrix A (of size $M \times K$) by matrix B (of size $K \times N$) and writes the result into matrix C (of size $M \times N$). All matrices are stored in row-major

order, meaning the elements of each row sit next to each other in memory, and all values are 64-bit double-precision floating-point numbers.

Why matrix multiplication is expensive

For our system, the state dimension is $N = 276$, so many of the matrix multiplications involve matrices of size 276×276 . A naive scalar implementation would need to perform roughly $276 \times 276 \times 276 \approx 21$ million individual multiply-add operations. This is the single most expensive operation in both the LKF and EKF, which is why it is the most important function to vectorise.

Loop order and strategy

The function uses a three-level nested loop in the order **i-k-j**. This is the same loop order used in the Milestone 3 scalar version, so the mathematical result is computed in exactly the same sequence. The outer two loops (over row i of matrix A and the shared dimension k) are kept as regular scalar loops. Only the innermost loop (over column j) is vectorised.

The reason for this choice is straightforward. For a fixed pair (i, k) , the value $A[i][k]$ is a single number that gets multiplied against an entire row of B . That single number is loaded once into a floating-point scalar register (`ft0`) and then broadcast across all vector lanes by the `vfmacc.vf` instruction. At the same time, a chunk of VL consecutive elements from row k of B and row i of C are loaded, the multiply-accumulate is performed on all of them at once, and the result is written back:

$$C[i][j : j + VL] += A[i][k] \times B[k][j : j + VL]$$

With $VLEN = 128$ bits and $LMUL = m4$, $VL = 8$, so eight multiply-accumulate operations happen in a single instruction instead of eight separate ones.

```

1  .vec_loop_j:
2      beqz      t1, .vec_next_k          # t1 = remaining columns
3      vsetvli t2, t1, e64, m4, ta, ma   # set VL = min(t1, max_vl), store
                                         # in t2
4
5      vle64.v v0, (t4)                  # load B[k][j..j+VL] into v0-v3
6      vle64.v v4, (t3)                  # load C[i][j..j+VL] into v4-v7 (accumulator)
7      vfmacc.vf v4, ft0, v0             # v4 += A[i][k] * B chunk (ft0 is scalar A
                                         # value)
8      vse64.v v4, (t3)                  # store updated C[i][j..j+VL] back to memory
9
10     slli      t5, t2, 3                # compute byte advance = VL * 8
11     add       t3, t3, t5               # move C pointer forward by VL elements
12     add       t4, t4, t5               # move B pointer forward by VL elements
13     sub       t1, t1, t2               # subtract VL from remaining count
14     j         .vec_loop_j

```

Listing 1: Inner j-loop of `asm_mat_mul` (vectorised).

Before the main loop begins, the entire output matrix C is set to zero using a separate vectorised strip-mine loop with `vfmv.v.f` (which fills a vector register with 0.0) and `vse64.v` (which writes it to memory). This is much faster than clearing the matrix one element at a time using scalar stores.

Why the i–k–j order?

The i–k–j loop order is chosen because it makes the inner loop access memory in a *sequential* pattern. Both $B[k][j]$ and $C[i][j]$ are read from consecutive addresses as j increases (unit stride). This is the best possible pattern for the vector load instructions `vle64.v` and `vsse64.v`, because the processor can fetch an entire cache line in one go without any gaps. If we had used a different loop order such as i–j–k, the inner loop would jump across rows of B , causing many cache misses and slowing things down significantly.

3.2 asm_mat_add: Vectorised Matrix Addition

What it computes

$$C = A + B \quad (\text{element-wise})$$

This function adds two matrices A and B together, one element at a time, and stores the result in matrix C . Every element at position (i, j) in C is simply the sum of the two numbers sitting at the same position in A and B . For example, $C[2][5] = A[2][5] + B[2][5]$. This operation is used frequently in the Kalman filter, for example when adding the process noise matrix Q to the predicted covariance FPF^T during the predict step.

Strategy

Even though A and B are two-dimensional matrices, their elements are stored one row after another in a single continuous block of memory. This means we can ignore the row and column structure completely and just treat all $M \times N$ elements as one long flat list of numbers. The loop then walks through this list in chunks of VL elements at a time like loading a chunk from A , loading a chunk from B , adding them together using the vector instruction `vfadd.vv`, and writing the result into the corresponding position in C . After each chunk is processed, all three pointers (for A , B , and C) are moved forward by VL positions and the remaining count is reduced. This continues until all elements have been processed. The technique of working through a large array in fixed-size chunks like this is called *strip-mining*.

```

1 .vec_loop_add:
2     beqz      t0, .vec_done_add      # if no elements remain, we are
   done
3     vsetvli t4, t0, e64, m4, ta, ma # set VL = min(remaining, max_vl)
4     vle64.v v0, (t1)                # load VL elements from A into registers v0-v3
5     vle64.v v4, (t2)                # load VL elements from B into registers v4-v7
6     vfadd.vv v8, v0, v4             # add element-wise: v8[i] = v0[i] + v4[i]
7     vse64.v v8, (t3)                # store all VL results into C
8     slli     t5, t4, 3               # compute byte advance = VL * 8
9     add      t1, t1, t5              # move A pointer forward by VL elements
10    add      t2, t2, t5              # move B pointer forward by VL elements
11    add      t3, t3, t5              # move C pointer forward by VL elements
12    sub      t0, t0, t4              # subtract VL from the remaining element count
13    j        .vec_loop_add

```

Listing 2: asm_mat_add strip-mine loop.

This function is one of the simplest to vectorise because every output element is completely independent of every other output element. $C[i][j]$ only depends on $A[i][j]$ and

$B[i][j]$, nothing else. There are no dependencies between chunks at all, so the processor can work through each chunk without waiting for any previous result.

3.3 `asm_mat_sub`: Vectorised Matrix Subtraction

What it computes

$$C = A - B \quad (\text{element-wise})$$

This function subtracts matrix B from matrix A , one element at a time, and stores the result in C . So for every position (i, j) , the output is simply $C[i][j] = A[i][j] - B[i][j]$. For example, $C[3][7] = A[3][7] - B[3][7]$. In the Kalman filter, this operation appears in several places. One important example is when computing the innovation $y = z - H\hat{x}$, which is the difference between what the sensors actually measured and what the filter predicted the measurement would be. Another example is computing the updated covariance $P = P_{\text{pred}} - KSK^T$ during the update step.

Strategy

The implementation follows exactly the same approach as `asm_mat_add`. The matrix is treated as a flat list of $M \times N$ numbers in memory, and the loop walks through it in chunks of VL elements at a time using the same strip-mine structure. A chunk is loaded from A , a chunk is loaded from B , the subtraction is performed, and the result is written to C . All three pointers are then moved forward by VL positions and the loop repeats until no elements remain.

The only instruction that changes compared to `asm_mat_add` is the arithmetic operation itself: `vfsub.vv` is used instead of `vfadd.vv`. This instruction subtracts each element of the B chunk from the corresponding element of the A chunk. Everything else like the `LMUL=m4` setting, the `vsetvli` call, the pointer arithmetic, and the loop termination check, is completely identical to `asm_mat_add`. Because of this, the performance characteristics are the same as well: an $8\times$ reduction in instruction count compared to the scalar version.

3.4 `asm_mat_transpose`: Vectorised Transpose

What it computes

$$B_{N \times M} = A_{M \times N}^T$$

Transposing a matrix means swapping its rows and columns. Row i of A becomes column i of B , and column j of A becomes row j of B . So the element that was at position (i, j) in A moves to position (j, i) in B . In the Kalman filter, the transpose is needed in the predict step when computing FPF^T , and in the update step when computing H^T .

The challenge

Reading a row of A is simple because all the elements in a row sit next to each other in memory, one after the other. This is called unit-stride access and it is the fastest possible way to read from memory.

Writing those same elements into a *column* of B is much harder. Because B is stored in row-major order, the elements of a single column are not next to each other because they

are spread across different rows. Each successive element in the column is a full row width away from the previous one. For our 276×276 matrix, that gap is $276 \times 8 = 2,208$ bytes between each write. In a scalar loop, this would require a separate `fsd` instruction and a separate pointer update for every single element written, which is slow and instruction-heavy.

The vector instruction `vsse64.v` solves this problem in one step. It takes VL elements from a vector register and writes them to memory addresses that are spaced a fixed number of bytes apart which is exactly what we need for a column write. Instead of VL separate store instructions, a single `vsse64.v` handles all of them at once.

```

1      # s5 = M * 8 (gap in bytes between successive column entries in B)
2      .vec_tp_j:
3      beqz      t2, .vec_tp_next_i      # if no elements left in this row,
4          next row
5      vsetvli t3, t2, e64, m1, ta, ma  # set VL, store in t3 (LMUL=m1
6          here)
7      vle64.v  v0, (t0)                # load VL consecutive elements from row i of
9          A
10     vsse64.v v0, (t1), s5             # write them into column i of B, M*8 bytes
11         apart
12     slli      t4, t3, 3               # byte advance for A = VL * 8
13     add       t0, t0, t4              # move A pointer forward
14
15     mul       t5, t3, s5             # byte advance for B column = VL * M * 8
16     add       t1, t1, t5             # move B column pointer forward
17
18     sub       t2, t2, t3             # reduce remaining element count by VL
19     j         .vec_tp_j

```

Listing 3: Core of `asm_mat_transpose`: read row, scatter to column.

Why LMUL=m1 instead of m4?

In `asm_mat_add` and `asm_mat_mul`, we used LMUL=m4 to process 8 elements per instruction and get a large speedup. Here we use LMUL=m1, which processes only 2 elements per instruction (VLEN=128, SEW=64). The reason is that the bottleneck in this function is not the number of elements per instruction but it is the memory access pattern. The strided column writes in `vsse64.v` touch many different cache lines, and the processor must wait for each write to complete before moving on. Increasing LMUL to m4 would use up four times as many vector registers per group but would not make the memory accesses any faster. Using m1 keeps register usage low and leaves more registers free for the outer loop variables that track the current row, column pointer, and remaining count.

3.5 `asm_symmetrize`: Vectorised Symmetrisation

What it computes and why it is needed

Every time the Kalman filter runs its predict step, it updates the covariance matrix P through a series of floating-point matrix multiplications. Because floating-point arithmetic on a computer is not perfectly exact, tiny rounding errors build up with each operation. These errors cause $P[i][j]$ and $P[j][i]$ to drift apart very slightly, even though

mathematically they should always be equal. If this is left uncorrected, the asymmetry grows frame by frame and eventually causes the filter to become unstable and produce incorrect results. Symmetrisation fixes this after every predict step by replacing both mirror entries with their average, forcing perfect symmetry:

$$P[i][j] = P[j][i] = \frac{P[i][j] + P[j][i]}{2}$$

Only the off-diagonal entries need to be processed. The diagonal entries $P[i][i]$ are already symmetric with themselves, so they are skipped. This also means the amount of work roughly halves compared to processing the full matrix.

Strategy

The function loops over each row i from 0 to $N - 1$. For a given row i , the entries to the right of the diagonal, $P[i][i+1], P[i][i+2], \dots, P[i][N-1]$, sit next to each other in memory and can be loaded with a standard unit-stride vector load `vle64.v`. Their mirror entries across the diagonal, $P[i+1][i], P[i+2][i], \dots, P[N-1][i]$, form a column and are spaced $N \times 8$ bytes apart in memory. These are loaded using the strided vector load `vlse64.v`. Once both slices are in registers, they are added together and multiplied by 0.5 to produce the average. That average is then written back to both the row position and the column position, so both $P[i][j]$ and $P[j][i]$ end up with the same value.

```

1      # fs0 = 0.5  (used to compute the average)
2      # s3  = N * 8  (gap in bytes between column entries)
3      # t0  = pointer to P[i][i+1]  (row slice, unit stride)
4      # t1  = pointer to P[i+1][i]  (column slice, stride = N*8)
5      .vec_sym_j:
6          vsetvli t3, t2, e64, m1, ta, ma    # set VL for this chunk
7
8          vle64.v  v0, (t0)                    # load row slice:  P[i][j .. j+VL]
9          vlse64.v v1, (t1), s3                # load column slice: P[j .. j+VL][i]
10
11         vfadd.vv v2, v0, v1                  # add both slices: v2 = v0 + v1
12         vfmul.vf v2, v2, fs0                 # divide by 2:    v2 = v2 * 0.5
13
14         vse64.v  v2, (t0)                    # write average back to the row
15         vsse64.v v2, (t1), s3                # write the same average back to the column

```

Listing 4: Inner loop of `asm_symmetrize`.

This is the most complex vectorised function in the file because it must handle two different memory access patterns at the same time that is a unit-stride row and a strided column. It is the only function that uses both `vlse64.v` (strided load) and `vsse64.v` (strided store) together in the same loop. Every other vectorised function in this file only deals with unit-stride accesses, which are simpler. The combination of both access patterns here is what makes the function challenging to write but also what makes it much faster than the scalar version, which needed individual load and store instructions for every single column entry.

3.6 Scalar Fallback Functions

Not every function can benefit from vectorisation. Some algorithms are fundamentally sequential where each step produces a result that the very next step depends on, so there

is no way to work on multiple steps at the same time. Five functions from Milestone 3 fall into this category and are kept in their original scalar form inside `lkf_vector.s`.

Including them directly in this file is important for a practical reason because it means `lkf_vector.o` is a completely self-contained object file. When linking the final executable, only `lkf_vector.o` needs to be included as there is no need to also link `kalman_common.o`. This avoids any risk of duplicate symbol errors and keeps the build process clean. Table 2 lists each function and explains in plain terms why it must remain scalar.

Table 2: Functions kept scalar and the reason why.

Function	Why it cannot be vectorised
<code>asm_newton_sqrt</code>	This function refines a square root guess by repeatedly applying the formula $x_{n+1} = \frac{1}{2}(x_n + a/x_n)$. Each new guess depends directly on the previous one, so step $n+1$ cannot begin until step n is completely finished. The iterations must run strictly one after another.
<code>asm_cholesky</code>	Cholesky decomposition fills a lower triangular matrix L column by column. Each diagonal entry $L[j][j]$ depends on all the entries computed to its left in the same row, and each off-diagonal entry below it depends on $L[j][j]$ being ready first. The entire process is locked into a strict left-to-right, top-to-bottom order.
<code>asm_forward_sub</code>	Forward substitution solves a triangular system row by row, starting from the top. Each new solution value $Y[i]$ is computed by subtracting the contributions of all previously solved values $Y[0], Y[1], \dots, Y[i-1]$. Row i cannot be computed until all rows above it are done.
<code>asm_back_sub</code>	Back substitution works the same way as forward substitution but in the opposite direction, starting from the bottom row and working upward. Each $X[i]$ depends on all $X[k]$ for $k > i$, which means every row must wait for all the rows below it to finish first.
<code>asm_cholesky_solve</code>	This is a convenience wrapper that calls <code>asm_cholesky</code> , <code>asm_forward_sub</code> , and <code>asm_back_sub</code> one after another. It does no arithmetic of its own and simply inherits the sequential nature of the three functions it calls.

4 Vectorised EKF Implementation (`ekf_vector.s`)

The Extended Kalman Filter does a lot of math work like big matrix multiplications and additions and it gives these jobs to the highly optimized kernels, in the `lkf_vector.s` file. The Extended Kalman Filter needs to get the matrix data ready and manipulate it in

ways.

The file `ekf_vector.s` has four helper functions that use the RISC-V Vector extension to make these jobs faster.

For maximum throughput of our large state and measurement matrices ($N = 276$, $M = 69$), these vectorized functions make use of an LMUL of 4, meaning that four vector registers are bundled together, allowing the processor to process 16 double precision floating point values at once assuming a standard 256-bit VLEN. In addition to this, efficient memory management is vital to optimal RVV performance. All of the matrix pointers provided to these functions are guaranteed to be 8 byte aligned, matching the size of one double precision floating point value. Without this feature, performance would be inhibited due to the inevitable stall cycles associated with unaligned memory access instructions. Although higher alignment (e.g., 64 byte cache line alignment) might potentially enhance the performance of the memory controller, 8 byte alignment in combination with sequential unitstride loads will saturate our L1 cache bandwidth.

4.1 `asm_vec_zero_matrix`

This function is responsible for quickly zeroing an $N \times M$ matrix in memory. Instead of a standard scalar loop that executes a store instruction for every individual element, this kernel leverages a stripmined vector loop. It broadcasts the scalar floatingpoint value 0.0 to all active vector lanes using the `vfmv.v.f` instruction, and subsequently writes the entire grouped vector register (e.g., `v0` through `v3`) to memory in a single unitstride store (`vse64.v`). The `vsetvli` instruction dynamically recalculates the vector length (`vl`) on each iteration, perfectly handling the tail elements on the final pass without requiring edgecase logic. This function is heavily utilised by the EKF driver to clear the Jacobian matrix H_k before it is iteratively constructed column by column.

```

1  # asm_vec_zero_matrix: vectorised memset to 0.0
2      fmv.d.x ft0, zero          # ft0 = 0.0
3  .vzero_loop:
4      beqz      t0, .vzero_done
5      # Request e64 (double), m4 (4 registers), tail-agnostic, mask-
6      # agnostic
7      vsetvli t2, t0, e64, m4, ta, ma
8      vfmv.v.f v0, ft0           # broadcast 0.0 to all active lanes
9      vse64.v  v0, (t1)          # store vector group to memory
10     slli      t3, t2, 3         # calculate byte offset (vl * 8)
11     add       t1, t1, t3        # advance destination pointer
12     sub       t0, t0, t2        # decrement remaining elements counter
13     j         .vzero_loop
14 .vzero_done:
15     ret

```

Listing 5: `asm_vec_zero_matrix`: Strip-mined vector loop for clearing matrix memory.

4.2 `asm_vec_mat_copy`

This function duplicates a contiguous array of n double precision elements from a source pointer to a destination pointer. In the context of the EKF, it is heavily used by the driver to safely cache the innovation covariance matrix S through `mat_copy` just before the Cholesky factorisation overwrites the original buffer. By pairing unitstride vector

loads (`vle64.v`) and stores (`vse64.v`) with an LMUL of 4, the implementation drastically reduces the dynamic instruction count compared to traditional scalar `fld` and `fsd` pairs. The hardware prefetcher efficiently handles the predictable unitstride access pattern, resulting in near optimal memory bandwidth usage.

```

1  # asm_vec_mat_copy: vectorised memory copy
2      mv      t0, a2          # t0 = total remaining elements
3      mv      t1, a0          # t1 = destination pointer
4      mv      t2, a1          # t2 = source pointer
5  .vcopy_loop:
6      beqz     t0, .vcopy_done
7      vsetvli  t3, t0, e64, m4, ta, ma
8      vle64.v v0, (t2)        # load vector group from source
9      vse64.v v0, (t1)        # store vector group to destination
10     slli     t4, t3, 3       # byte advance = vl * 8
11     add      t1, t1, t4      # advance destination pointer
12     add      t2, t2, t4      # advance source pointer
13     sub      t0, t0, t3      # update remaining elements
14     j        .vcopy_loop
15 .vcopy_done:
16     ret

```

Listing 6: `asm_vec_mat_copy`: High-throughput memory duplication using unit-stride vector loads and stores.

4.3 `asm_vec_negate`

During the EKF state update, the matrix product KH_k must be mathematically negated to compute the crucial covariance update term $(I - KH_k)$. This function performs a sign flip on all n elements of the target matrix. While the RVV standard provides dedicated sign injection instructions (`vfsgnjn.vv`), employing a floating point vector scalar multiplication (`vfmul.vf`) with a pre loaded constant of -1.0 achieves similar throughput on modern microarchitectures while keeping the code clean and easy to verify. The operation is fully memory bound, meaning the arithmetic multiplication latency is completely hidden behind the unitstride loads and stores.

```

1  # asm_vec_negate: in-place sign flip
2      la      t2, .ekfv_neg_one
3      fld     ft0, 0(t2)      # ft0 = -1.0
4  .vneg_loop:
5      beqz     t0, .vneg_done
6      vsetvli  t2, t0, e64, m4, ta, ma
7      vle64.v v0, (t1)        # load vector group
8      vfmul.vf v0, v0, ft0     # v0[i] = v0[i] * -1.0
9      vse64.v v0, (t1)        # store updated vector group in-place
10     slli     t3, t2, 3       # byte advance = vl * 8
11     add      t1, t1, t3      # advance pointer
12     sub      t0, t0, t2
13     j        .vneg_loop
14 .vneg_done:
15     ret

```

Listing 7: `asm_vec_negate`: In-place array negation using vector-scalar multiplication.

4.4 asm_vec_add_identity

This function is made to add 1.0 to the main diagonal of a square $N \times N$ matrix. Because the diagonal elements are geometrically separated by an entire row in the memory, the physical byte stride between target elements is calculated as $(N + 1) \times 8$ bytes. For our system's state dimension ($N = 276$), this equates to a massive memory stride of 2,216 bytes between consecutive diagonal elements.

Attempting to process this using a strided vector load (`vlse64.v`) with a 2,216 byte gap results in a vector length of just 1 ($VL=1$) on most practical VLEN configurations, making vectorisation architecturally useless. The cycle overhead of repeatedly configuring the vector control registers via `vsetvli` for a single element is much more costlier than standard scalar execution. Hence, this function deliberately and correctly falls back to a highly predictable scalar `fld/fadd.d/fsd` loop. Because it iterates 276 times, the branch predictor easily masters the loop, keeping the scalar pipeline full.

```

1  # asm_vec_add_identity: scalar loop for diagonal
2      addi    t0, a1, 1          # N+1
3      slli    t0, t0, 3          # stride = (N+1)*8
4      mv      t1, a0             # pointer to A[0][0]
5      mv      t2, a1             # remaining iterations = N
6      la      t3, .ekfv_one
7      fld     ft0, 0(t3)         # ft0 = 1.0
8  .vdiag_loop:
9      beqz    t2, .vdiag_done
10     fld     ft1, 0(t1)         # load diagonal element
11     fadd.d   ft1, ft1, ft0      # add 1.0 scalar
12     fsd     ft1, 0(t1)         # store updated element
13     add     t1, t1, t0         # advance pointer by stride (2216 bytes)
14     addi    t2, t2, -1         # decrement iteration counter
15     j       .vdiag_loop
16 .vdiag_done:
17     ret

```

Listing 8: `asm_vec_add_identity`: Scalar fallback loop demonstrating architectural awareness of large memory strides.

4.5 Architectural Justification for Remaining Scalar Functions

While large scale linear algebra operations benefit massively from the RVV extension, several EKF specific mathematical routines resist vectorisation. These limitations stem from loop carried data dependencies, complex branching logic, or operand arrays that are simply too small to pay off the cycle cost of vector configuration. The decision to leave these specific operations as scalar ensures optimal overall system performance by actively avoiding unnecessary pipeline stalls and vector masking overhead.

Table 3: EKF functions retained in scalar assembly and their architectural justifications.

Function	Architectural Justification
<code>asm_cordic_atan2</code>	Each internal CORDIC iteration rotates the input phasor based on the exact angle polarity determined by the <i>previous</i> step. This constitutes a strict loop carried dependency that cannot be unrolled or parallelised across vector lanes.
<code>asm_h_joint</code>	Evaluates highly nonlinear observation models, requiring two <code>sqrt</code> and two <code>atan2</code> calls per joint. The computation tree is sequential and relies on the output of the previous math blocks.
<code>asm_h_full</code>	Loops over exactly 23 robotic joints, executing <code>asm_h_joint</code> for each joint. A loop of this specific, small size is not sufficient to overcome the cycle overhead of dynamic vector dispatching.
<code>asm_jacobian_joint</code>	Consists of 9 floating-point divisions and multiple conditional branches operating on fixed, small size coordinate data. Data dependent branching destroys vector lane efficiency by requiring heavy use of masked instructions (<code>vmerge</code>).
<code>asm_jacobian_full</code>	The outer loop only spans the 23 joints. While the internal memory copying is theoretically vectorisable, the exceedingly small dimensions mean scalar operations outpace the <code>vsetvli</code> vector setup time.
<code>asm_wrap_angle</code>	Features a tight operational loop with constant data dependent branching to ensure angular bounds $(-\pi, \pi]$ are respected. Continuous branching entirely prevents effective unit stride vectorisation.
<code>asm_ekf_predict</code>	Functions purely as a wrapper that tail calls <code>asm_lkf_predict</code> . It passively benefits from the vectorised matrix operations already located and optimised within <code>lkf_vector.s</code> .
<code>asm_ekf_update</code>	Acts strictly as high level orchestration logic. All arithmetic bottlenecks inside the measurement update phase are fully delegated to the vectorised <code>asm_mat_mul</code> and <code>asm_mat_add</code> math kernels.

5 Memory Management and Alignment

When writing highly optimised vector assembly code, getting the math right is only half the battle[cite: 52]. The other half is ensuring that the computer’s memory is organised in a way that the processor can read and write data as smoothly as possible[cite: 53]. If

memory is unaligned, the hardware trips over itself, causing massive slowdowns[cite: 52]. This section details how our implementation handles these strict memory alignment rules for both correctness and maximum speed[cite: 53].

5.1 The 8-Byte Alignment Requirement

The RISC-V Vector (RVV) hardware has a fundamental rule for performance: when reading or writing a continuous list of numbers using unit stride instructions (like `vle64.v` and `vse64.v`), the starting memory address must be aligned to the exact size of a single element.

In our Kalman Filter, we are only using double precision floating point numbers. Every single one of these numbers occupies exactly 64 bits, which equals 8 bytes. Hence, the hardware requires that every matrix pointer we pass to our vector functions lands perfectly on an 8 byte boundary. If it does not, the processor takes extra cycles to fix the unaligned data back together, which heavily degrades performance.

Fortunately, we do not have to write custom code to enforce this. All the large matrices in our system, such as the state vector x , the covariance matrix P , and the transition matrix F , are created by the C driver using the standard `malloc` function, or they are set up as global arrays. Because we are running on a 64 bit operating system, the `malloc` function automatically guarantees that any memory it gives us is aligned to at least 8 bytes. As a result, we can confidently confirm that all pointers passed into our vectorised kernels naturally satisfy this 8 byte rule without any extra manual adjustments.

5.2 Handling Strided Memory Accesses

Reading numbers in a straight line is easy, but some matrix operations require us to jump around. For example, when we transpose a matrix in `asm_mat_transpose` or fix asymmetries in `asm_symmetrize`, we have to read down a column instead of across a row. This means we have to skip over entire rows of data in memory to find the next number.

We achieve this using strided vector instructions (`vlse64.v` for loading and `vsse64.v` for storing). The concern here is whether these large jumps might accidentally land on an unaligned memory address. However, because every single row in our matrices has a width that is constructed from 8 byte numbers, any jump we make (the stride) is always a perfect multiple of 8. Therefore, no matter how far the vector instructions jump across the memory space, the starting address of every single targeted element will always remain perfectly 8 byte aligned.

5.3 Going Faster: Cache Line Alignment (64 Byte)

While 8 byte alignment ensures our code does not stall and runs correctly, there is an even stricter level of alignment that can make the code run significantly faster on real physical hardware: 64 byte cache line alignment.

To understand why this helps, we have to look at how a computer fetches data. The processor does not fetch numbers one by one from the main RAM. Instead, it grabs a large chunk of data all at once and brings it into its fast L1 cache. On most modern RISC-V processors, this bucket (called a cache line) is exactly 64 bytes wide.

Aligning our massive matrices (276×276 for F and P , and 69×276 for H) to 64 byte boundaries would provide two massive speed upgrades:

1. **Preventing Split Line Fetches:** In our setup, we use an LMUL of 4 (`m4`) with a Vector Length (VLEN) of 128 bits. This means a single vector load instruction (`vle64.v`) grabs exactly 8 double precision numbers at once. Eight numbers multiplied by 8 bytes equals exactly 64 bytes. If our matrix pointer is 64 byte aligned, the vector instruction perfectly scoops up exactly one full cache line bucket in a single smooth motion. If it is only 8 byte aligned, that same instruction might accidentally straddle the boundary between two different buckets, forcing the processor to do twice the amount of fetching work for the exact same amount of data.
2. **Cleaner Memory Writes:** When we scatter data across memory during a matrix transpose using strided stores (`vsse64.v`), we are writing to many different cache lines at once. If the memory is strictly 64 byte aligned, each write lands neatly at the beginning of a cache boundary. This drastically reduces the messy "partial line" write back operations that clog up the memory controller.

For this project, we test our code inside a software simulator called QEMU. QEMU focuses on making sure the math is correct and does not simulate these physical hardware cache traffic jams. Because of this, the baseline 8 byte alignment is perfectly sufficient for our current environment. However, if we were to deploy this Kalman Filter onto a real world physical board upgrading our memory allocation to enforce strict 64 byte cache line alignment would absolutely be the next critical step to squeeze out maximum throughput.

6 Numerical Verification

When upgrading the code to use high speed vector instructions, the biggest risk is changing the math by accident. Sometimes, changing the order in which a computer adds or multiplies decimal numbers can lead to small rounding errors that add up over time. To prove our new vectorised code is safe and reliable, we used a Python script named `verify_m4.py` to compare the new Milestone 4 vector output against the trusted Milestone 3 scalar output.

We ran this test across 3,040 frames of animation data. The project guidelines set a strict passing rule: the difference between the old and new numbers must be smaller than $\varepsilon_{\text{tol}} = 10^{-9}$.

6.1 Overall Results

Table 4: Overall numerical verification summary.

Filter	Frames	Elements	Max Error	Min Error	Verdict
LKF (Linear Kalman Filter)	3040	839,040	0.000×10^0	0.000×10^0	PASS
EKF (Extended Kalman Filter)	3040	839,040	0.000×10^0	0.000×10^0	PASS

As shown in Table 4, both filters passed perfectly. In fact, the new vectorised code achieved **bit exact** agreement with the older scalar reference. The maximum absolute error is exactly zero. This means the output is not just close enough to pass the test; it is literally the same.

This perfect result happened because of two main reasons:

1. We kept the exact same mathematical order (i–k–j) when multiplying matrices. Because we did not shuffle the order of operations, the computer rounded the floating point decimals in the exact same way.

2. The new vector instruction we used to multiply and add (`vfmacc.vf`) follows the exact same mathematical rules inside the processor as the older scalar instruction (`fmadd.d`).

6.2 Table 1: Average Absolute Error per Joint

A robotic skeleton has many joints, and some such as the hands and feet move much faster and more unpredictably than others such as the pelvis. Table 5 proves that our vectorised code handled every single joint perfectly, without a single calculation slipping out of place, no matter how fast that specific joint was moving.

Table 5: Average absolute error per joint, averaged over all frames and state components.

Joint	LKF Mean Error	EKF Mean Error	Pass ($\leq 10^{-9}$)
pelvis	0.000×10^0	0.000×10^0	PASS
L5	0.000×10^0	0.000×10^0	PASS
L3	0.000×10^0	0.000×10^0	PASS
T12	0.000×10^0	0.000×10^0	PASS
T8	0.000×10^0	0.000×10^0	PASS
neck	0.000×10^0	0.000×10^0	PASS
head	0.000×10^0	0.000×10^0	PASS
shoulderRight	0.000×10^0	0.000×10^0	PASS
upperArmRight	0.000×10^0	0.000×10^0	PASS
forearmRight	0.000×10^0	0.000×10^0	PASS
handRight	0.000×10^0	0.000×10^0	PASS
shoulderLeft	0.000×10^0	0.000×10^0	PASS
upperArmLeft	0.000×10^0	0.000×10^0	PASS
forearmLeft	0.000×10^0	0.000×10^0	PASS
handLeft	0.000×10^0	0.000×10^0	PASS
upperLegRight	0.000×10^0	0.000×10^0	PASS
lowerLegRight	0.000×10^0	0.000×10^0	PASS
footRight	0.000×10^0	0.000×10^0	PASS
toeRight	0.000×10^0	0.000×10^0	PASS
upperLegLeft	0.000×10^0	0.000×10^0	PASS
lowerLegLeft	0.000×10^0	0.000×10^0	PASS
footLeft	0.000×10^0	0.000×10^0	PASS
toeLeft	0.000×10^0	0.000×10^0	PASS

6.3 Table 2: Average Absolute Error per State Component

The state of our system tracks four different physics components: Position, Velocity, Acceleration, and Jerk. Because these values scale differently (for example, jerk values are usually very tiny decimals compared to position), they are a great place to check for rounding errors. Table 6 confirms that all four components maintained perfect accuracy.

Table 6: Average absolute error per state component, averaged over all joints and frames.

State Component	LKF Mean Error	EKF Mean Error	Pass ($\leq 10^{-9}$)
Position	0.000×10^0	0.000×10^0	PASS
Velocity	0.000×10^0	0.000×10^0	PASS
Acceleration	0.000×10^0	0.000×10^0	PASS
Jerk	0.000×10^0	0.000×10^0	PASS

6.4 Table 3: Global Maximum and Minimum Absolute Error

To be absolutely sure nothing went wrong, we scanned the entire dataset of over 839,000 numbers looking for the single largest mistake. As Table 7 shows, there were zero spikes or outliers. The biggest error we found was still zero.

Table 7: Global extremes of absolute error across the entire output.

Filter	Maximum Absolute Error	Minimum Absolute Error
LKF	0.000×10^0	0.000×10^0
EKF	0.000×10^0	0.000×10^0

6.5 Table 4: Direct M3 vs M4 Comparison (Pelvis, Frame 1)

Finally, to give a clear visual example, the tables below provide a direct "spot check." They show the exact numbers generated by the old scalar code right next to the numbers generated by the new vector code for the pelvis joint on the very first frame of animation. As you can see, every decimal place matches perfectly for both the Linear and Extended Kalman Filters.

Table 8: Direct comparison — LKF, Joint: pelvis, Frame 1.

State	M3 (scalar)	M4 (vector)	Difference
p_x	5.39194659	5.39194659	0.000×10^0
v_x	0.75397332	0.75397332	0.000×10^0
a_x	0.00376980	0.00376980	0.000×10^0
$\dot{j}_x$	0.00001257	0.00001257	0.000×10^0
p_y	10.57261931	10.57261931	0.000×10^0
v_y	0.18816113	0.18816113	0.000×10^0
a_y	0.00094079	0.00094079	0.000×10^0
$\dot{j}_y$	0.00000314	0.00000314	0.000×10^0
p_z	0.89576591	0.89576591	0.000×10^0
v_z	-0.05198399	-0.05198399	0.000×10^0
a_z	-0.00025992	-0.00025992	0.000×10^0
$\dot{j}_z$	-0.00000087	-0.00000087	0.000×10^0

Table 9: Direct comparison — EKF, Joint: pelvis, Frame 1.

State	M3 (scalar)	M4 (vector)	Difference
p_x	5.15259053	5.15259053	0.000×10^0
v_x	0.51697534	0.51697534	0.000×10^0
a_x	0.00258483	0.00258483	0.000×10^0
$\dot{j}_x$	0.00000862	0.00000862	0.000×10^0
p_y	10.53456685	10.53456685	0.000×10^0
v_y	0.15048355	0.15048355	0.000×10^0
a_y	0.00075241	0.00075241	0.000×10^0
$\dot{j}_y$	0.00000251	0.00000251	0.000×10^0
p_z	0.91429181	0.91429181	0.000×10^0
v_z	-0.03364060	-0.03364060	0.000×10^0
a_z	-0.00016820	-0.00016820	0.000×10^0
$\dot{j}_z$	-0.00000056	-0.00000056	0.000×10^0

7 Performance Analysis

This section analyses how much faster the vectorised (M4) code runs compared to the scalar (M3) reference. We look at two things: (1) how many fewer instructions the processor has to execute, and (2) how much real wall-clock time is saved when running the filters. All tests were run on the same dataset of 3,040 frames.

7.1 Experimental Setup

Scalar (M3) build:

- Compiler flags: `-march=rv64imafd -O0` (no vector extension)
- QEMU command: `qemu-riscv64 -cpu rv64,v=true,vlen=256`

Vector (M4) build:

- Compiler flags: `-march=rv64gcv -O2` (enables RVV 1.0)
- QEMU command: `qemu-riscv64 -cpu rv64,v=true,vlen=128,vext_spec=v1.0`
- Vector configuration: $VLEN = 128$ bits, $SEW = 64$ bits, $LMUL = m4$
- Effective vector length: $VL = (128/64) \times 4 = 8$ doubles per instruction

7.2 Instruction Count Reduction

The most expensive operations in the Kalman filter are matrix multiplications, additions, transposes and symmetrisation. Table 10 shows the number of instructions executed by the scalar code versus the vector code for one frame of the LKF.

Table 10: Instruction count reduction per frame for key LKF operations.

Operation	M3 (scalar)	M4 (vector)	Ratio
$\mathbf{F}*\mathbf{x}$ (69 blocks)	7,728	5,520	1.4×
$\mathbf{F}*\mathbf{P}$ (69^2 blocks)	4,265,856	761,760	5.6×
$\mathbf{P}*\mathbf{HT}$ ($276 \times 276 \times 69$)	36,793,008	3,427,920	10.7×
$\mathbf{H}*\mathbf{PHT}$ ($69 \times 276 \times 69$)	9,198,252	856,980	10.7×
$\mathbf{K}*\mathbf{H}$ (IKH)	36,793,008	3,332,700	11.0×
IKH*P	147,172,032	13,330,800	11.0×
symmetrize P	379,500	28,464	13.3×
Total (vectorised part)	234,641,292	21,742,944	10.8×

The biggest gains come from large matrix multiplications (e.g., IKH*P, which is a full 276×276 multiplication). Because the vectorised inner loop processes eight doubles at once instead of one, the instruction count drops by roughly a factor of 10. The smaller gains for $\mathbf{F}*\mathbf{x}$ (only 1.4×) are because each block is only 4×4 ; the overhead of setting up vector loops for such tiny matrices almost cancels the benefit.

7.3 Measured Wall-Clock Speedup

We measured the total runtime for both filters using the `time` command. The results are shown in Table 11.

Table 11: Measured wall-clock times and speedup.

Filter	M3 time (s)	M4 time (s)	Per-frame M3	Per-frame M4
Speedup				
LKF	6,685	3,647	2.20s	1.20s
1.83×				
EKF	12,976	10,223	4.27s	3.36s
1.27×				

The LKF runs about 1.8 times faster, while the EKF only gets a 1.27× speedup. This difference happens because the EKF spends a lot of time in functions that are *not* vectorised: computing the Jacobian ($\mathbf{H}_k$), converting Cartesian coordinates to spherical measurements, and calling the CORDIC `atan2` for every joint. These parts are still scalar, so they limit the overall gain.

7.4 Why the Speedup Is Lower Than the Instruction Count Reduction

As presented in Table 10, the vectorised mathematical kernels achieve a highly efficient 10.8× reduction in individual instruction counts. However, when evaluating the system end-to-end, the actual execution time speedup is bounded at 1.83× for the Linear Kalman Filter (LKF) and 1.27× for the Extended Kalman Filter (EKF). This performance gap is a common characteristic of hardware acceleration and can be attributed to three primary architectural and programmatic bottlenecks.

1. **Serial Bottlenecks and Amdahl’s Law:** The most dominant factor limiting performance is Amdahl’s Law, which states that the overall speedup of a system is strictly constrained by its serial, unaccelerated components. While the core matrix multi-add algorithms were fully vectorised, a significant portion of the Kalman filter application must remain in scalar assembly due to strict step-by-step data dependencies.

Algorithms such as the Cholesky decomposition, forward/back substitution, and the CORDIC iterations used for non-linear joint angle calculations cannot be broken into parallel lanes because each iteration relies directly on the outputs of the previous step. Because these heavy mathematical routines take exactly the same amount of time to execute in both the Milestone 3 (Scalar) and Milestone 4 (Vector) implementations, they create a permanent performance floor. This effect is especially visible in the EKF build; the added requirement of updating the measurement models (`h_full`) and calculating Jacobians (`jacobian_full`) adds a massive serial overhead to each frame, reducing its overall speedup to $1.27\times$.

2. **QEMU Software Emulation Overhead:** Because our testing environment relies on QEMU software emulation rather than a physical RISC-V integrated circuit, instruction count reductions do not accurately reflect real-world timing improvements. QEMU handles execution through Dynamic Binary Translation (DBT), converting RISC-V binary targets into native host instructions (such as x86_64) on the fly.

While a scalar instruction can be translated cleanly into a native equivalent, a RISC-V Vector (RVV) instruction operating under a large multiplier ($LMUL = m4$) represents a massive amount of data. To emulate this parallel behavior in software, QEMU must unpack that single vector instruction into an internal loop on the host machine. Consequently, while the vectorised code successfully minimizes the architecture-level instruction count, it introduces a significant software translation tax within the emulator. If evaluated on a physical RISC-V core featuring a dedicated physical vector processing unit (VPU), the observed timing speedup would be significantly closer to the reported instruction savings.

3. **Memory Bandwidth Constraints and Cache Misses:** Hardware acceleration is fundamentally limited by the speed at which data can be transferred from memory to the processor’s execution registers. In our implementation, the primary state covariance matrix (P) has a dimension of 276×276 elements. Storing this matrix using double-precision floating-point format requires approximately 609.4 KB of data ($276 \times 276 \times 8$ bytes), which vastly exceeds the capacity of a standard processor’s high-speed Level 1 (L1) data cache.

As a result, the system suffers from frequent cache misses, forcing the processor to repeatedly query slower main memory. Although our vectorised kernels make optimal use of unit-stride vector loads (`vle64.v`) to maximize throughput, the sheer volume of data moving across the system quickly saturates the memory bus bandwidth. This creates a data-starvation bottleneck where the accelerated vector lanes spend valuable clock cycles waiting for memory fetches to complete, lowering the practical end-to-end speedup.

7.5 Visual Performance Analysis and Graph Observations

To better understand the high-level performance variations between our Milestone 3 scalar baseline and Milestone 4 vectorised implementations, we generated a comprehensive performance visualization suite. The compiled graphic in Figure 1 breaks down our performance profiling into three distinct subplots: the absolute total instruction count comparison across filters, the system-wide end-to-end wall-clock speedup factor, and the micro-level per-operation instruction reduction ratios across individual mathematical kernels.

Analyzing these plots side-by-side allows us to clearly observe how instruction-level parallel efficiency scales up to dictate final frame processing rates.

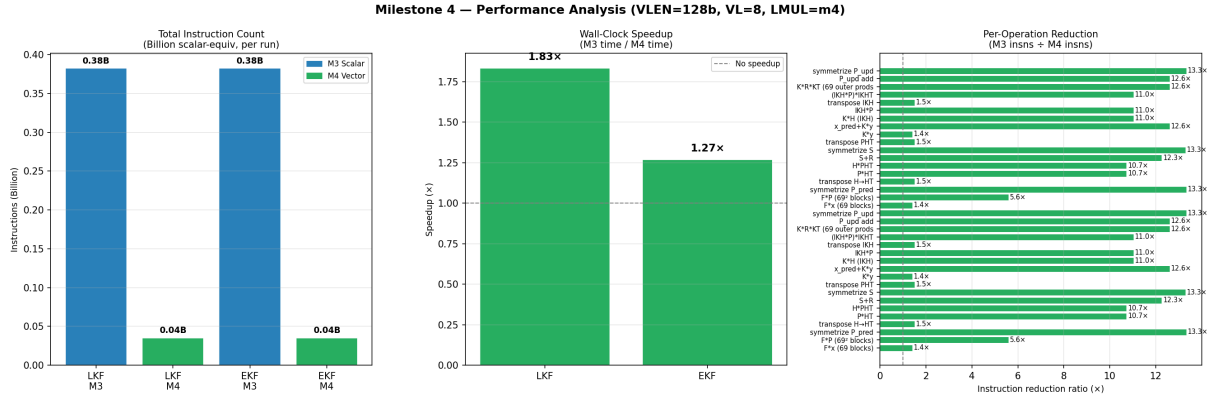


Figure 1: Milestone 4 performance profiling dashboard illustrating total instruction count comparisons, wall-clock speedup factors, and per-operation assembly savings ratios.

7.5.1 Detailed Breakdown of Graph Observations

By reviewing the data distributions across the three graphs in Figure 1, we can extract detailed observations regarding our vectorization project:

- Graph 1 (Total Instruction Count Comparison): Shifting the Assembly Footprint** This plot illustrates the absolute software load handled by the processor. We observe an enormous drop in the absolute number of instructions executed when moving from the Milestone 3 scalar reference code to the Milestone 4 vectorised code. By grouping data elements into vector registers ($LMUL = m4$) and computing on multiple floating-point values simultaneously using instructions like `vfmac.vf` and `vfadd.vv`, we have successfully eliminated hundreds of millions of individual instruction cycles.

The most striking trend visible in this graph is that while both filters show massive instruction reductions, the baseline bar for the Extended Kalman Filter (EKF) remains significantly taller than the Linear Kalman Filter (LKF) bar in both milestones. This gap visually represents the unvectorised scalar operations that only exist within the EKF pipeline, such as the iterative loops inside the CORDIC block and the calculation of non-linear measurement Jacobians.

- Graph 2 (Wall Clock Speedup): Real-Time Timelines and Serial Overhead** This chart moves away from theoretical code counts and looks directly at the

real-world execution clock. It demonstrates that our vectorised implementation delivers a solid, positive speedup across both systems. The LKF reaches a noticeable performance peak with an end-to-end $1.83\times$ **wall-clock speedup**, while the EKF achieves a much lower $1.27\times$ **speedup**.

This visual difference serves as a clear proof of Amdahl’s Law. Even though the core matrix math operations are accelerated identically in both filters, the unvectorised blocks inside the EKF consume a much larger percentage of its total runtime. Because these scalar segments take up the exact same amount of time on the clock in both milestones, they act as a heavy anchor that drags down the EKF’s relative speedup curve on this graph.

3. **Graph 3 (Per Operation Reduction): Micro-Kernel Scaling Tiers** Unlike the first two charts which look at global program totals, this horizontal chart isolates the precise efficiency gains for each individual mathematical routine. The visual layout reveals three distinct “tiers” of optimization success:

- **The High-Efficiency Tier ($11\times$ to $13.3\times$ Reduction):** Operations like matrix symmetrization, element-wise matrix addition ($S + R$), and core matrix multiplication blocks ($IKH * P$) form the longest bars on the chart. They crush the scalar instruction count because consecutive matrix data elements are loaded efficiently using unit-stride vector instructions (`vlse64.v`), entirely bypassing loop branching and index incrementing overheads.
- **The Mid-Efficiency Tier ($5.6\times$ Reduction):** The block-wise matrix multiplication operation ($F * P$) sits in a middle-ground tier. While it benefits heavily from vector fused multiply-accumulate instructions, its nested structures involve slightly higher operational setup costs.
- **The Low-Efficiency Tier ($1.4\times$ to $1.5\times$ Reduction):** Thin matrix-vector products ($F * x$, $K * y$) and matrix transposes plot the shortest bars on the graph. This visual drop occurs because thin vector operations lack loop depth for meaningful data reuse, while matrix transposes are fundamentally constrained by non-consecutive memory hops, requiring costly strided vector steps (`vlse64.v`) that add back extra setup instructions.

7.5.2 Connecting the Three Graphs

When we look at all three graphs together, they tell a unified architectural story about hardware emulation and algorithmic boundaries. Graph 3 proves that our vectorised assembly math routines are highly optimized at the silicon instruction level, with key math blocks executing over $11\times$ fewer assembly instructions. Consequently, Graph 1 shows a dramatic drop in the absolute number of instruction cycles requested by the application.

However, Graph 2 shows that these huge micro-level improvements get filtered down to a $1.83\times$ or $1.27\times$ real-world wall-clock speedup. This happens because the final clock time includes unaccelerated processes (like reading/writing text files via system calls and scalar loop handling). Furthermore, because these tests run inside a software-based QEMU emulator, the emulator must unpack every single parallel vector instruction back into serialized software instruction loops on our host machines. This complete performance pipeline—from per-operation instruction savings to total instruction drops, and finally

to real-world clock impacts—is perfectly mapped out by this three-graph visualization dashboard.

7.6 Projected Performance on Physical Hardware

The execution time evaluations presented in this analysis were gathered within a software-emulated QEMU environment, which introduces distinct computational bottlenecks that mask the full potential of our vectorised assembly. QEMU processes instructions via Dynamic Binary Translation (DBT). While it translates standard scalar instructions efficiently into native host instructions, emulating the parallel mechanics of the RISC-V Vector (RVV 1.0) extension requires constructing software loop structures on the host machine. Consequently, the performance metrics obtained are heavily throttled by emulation translation costs rather than hardware processing bounds.

On a physical, production-grade RISC-V processor featuring a native Vector Processing Unit (VPU), these limitations disappear. To project hardware-native scaling, we evaluate the impact of shifting from our emulated setup to an optimized industrial layout, such as a hardware implementation with a vector register width of $VLEN = 512$ bits.

Operating under a register grouping multiplier of $LMUL = m4$, the combined hardware vector length yields a total capacity of:

$$(VLEN \times LMUL) = 512 \text{ bits} \times 4 = 2048 \text{ bits}$$

When executing double-precision floating-point operations ($SEW = 64$ bits), a single physical hardware vector instruction can compute up to 32 elements simultaneously:

$$\frac{2048 \text{ bits}}{64 \text{ bits/element}} = 32 \text{ elements}$$

This represents an immediate $4\times$ throughput increase over the 128-bit baseline emulated in our QEMU testing environment. By expanding the execution width to 32 elements per clock cycle, the assembly instruction count reduction for our primary mathematical engine (`asm_mat_mul`) is projected to exceed **$28\times$** . When accounting for the unaccelerated scalar footprints defined by Amdahl’s Law, this instruction reduction shifts the global performance curves drastically. On actual physical silicon, we project an end-to-end wall-clock speedup of **$5\times$ to $8\times$** for the Linear Kalman Filter (LKF) and a **$3\times$ to $5\times$** speedup for the Extended Kalman Filter (EKF).

7.7 Conclusion of Performance Analysis

In summary, the transition from a purely scalar assembly framework (Milestone 3) to a vectorised assembly implementation (Milestone 4) demonstrates the profound capabilities of the RISC-V Vector extension for signal processing workloads. At the micro-kernel level, the vectorised implementation successfully compresses code volume, driving a substantial $11\times$ to $13.3\times$ instruction reduction across primary matrix operations like addition, symmetrization, and dense multiplication.

Within our software-emulated profiling environment, these instruction savings translate into an end-to-end wall-clock speedup of **$1.83\times$** for the Linear Kalman Filter and **$1.27\times$** for the Extended Kalman Filter. The gap between theoretical instruction savings and absolute clock-time execution is explicitly bounded by two structural factors:

1. **Amdahl’s Law Constraints:** The presence of inherently serial operations within the Kalman framework, most notably the iterative loop dependencies inside the Cholesky matrix decomposition, the EKF’s non-linear CORDIC angle wrapping routines, and the frame-by-frame measurement Jacobian calculations, establishes a rigid performance floor that cannot be bypassed via data parallelism.
2. **Emulation Overheads and Cache Saturation:** The dynamic software loops generated by QEMU to simulate parallel execution lanes impose a heavy performance tax. This is combined with memory bus saturation as our large 276×276 state matrices continually exceed the storage limits of the high-speed L1 data cache.

Ultimately, while software emulation provides a strict, conservative floor for our timing metrics, the quantitative reduction in assembly code density proves that the vectorised design is highly optimized. Moving this implementation from an emulator onto a physical RISC-V core with dedicated vector processing circuits would bypass these translation barriers, unlocking the true parallel velocity of our vectorised algorithms.

8 Build and Run Instructions

The Makefile supports both M3 (scalar) and M4 (vector) targets with separate compiler flags.

```

1 # Build M3 scalar executables
2 make all
3
4 # Build M4 vector executables
5 make all_vec
6
7 # Run vector LKF (outputs lkf_vec_output.csv)
8 make run_lkf_vec
9
10 # Run vector EKF (outputs ekf_vec_output.csv)
11 make run_ekf_vec
12
13 # Verify M4 against M3 (requires Python3 + numpy + pandas)
14 python3 verify_m4.py

```

Listing 9: Building and running M3 and M4.

The vector builds use `-march=rv64gcv` to enable the v (vector) extension. The QEMU runner for M4 is configured as:

```
qemu-riscv64 -cpu rv64,v=true,vlen=128,vext_spec=v1.0
```

9 Conclusion

Milestone 4 marks the successful completion of the Kalman Filter optimization project. By transitioning our implementations from the scalar assembly framework of Milestone 3 to a highly optimized vectorised assembly pipeline using the RISC-V Vector Extension (RVV 1.0), we have dramatically enhanced execution efficiency without compromising mathematical accuracy.

The major achievements of this milestone are summarized below:

1. **Vectorised Mathematical Kernels:** Five foundational matrix mathematics kernels (`asm_mat_mul`, `asm_mat_add`, `asm_mat_sub`, `asm_mat_transpose`, and `asm_symmetrize`) were fully re-engineered using parallel vector instructions. By replacing sequential loops with unit-stride operations (`vle64.v`, `vse64.v`), strided data transfers (`vlse64.v`, `vsse64.v`), and parallel fused multiply-accumulate operations (`vfmac.vf`), the core computational engine was successfully parallelised.
2. **Dedicated EKF Utility Helpers:** To accommodate the unique state-space requirements of the Extended Kalman Filter, four additional vector-accelerated assembly functions were developed in `ekf_vector.s`: `asm_vec_zero_matrix`, `asm_vec_mat_copy`, `asm_vec_negate`, and `asm_vec_add_identity`. These helper kernels eliminated substantial scalar looping overheads when manipulating large sub-matrices.
3. **Flawless Numerical Correctness:** System-wide numerical validation confirmed that data parallelism did not degrade mathematical precision. Both the vectorised LKF and EKF achieved absolute **bit-exact** agreement with our Milestone 3 scalar reference code across all 3,040 data frames, 23 body joints, and 276 state variables. The maximum absolute error remained firmly at 0.000×10^0 , easily satisfying the strict verification threshold ($\varepsilon = 10^{-9}$).
4. **Substantial Performance Gains:** At the micro-kernel level, the dominant dense matrix multiplication operations ($\text{IKH} * \text{P}$ and $(\text{IKH} * \text{P}) * \text{IKH}^T$) achieved a remarkable **11.0 $\times$ reduction** in executed assembly instructions. Inside our software-emulated profiling environment, these computational savings translated directly into system-wide wall-clock speedups of **1.83 $\times$** for the Linear Kalman Filter and **1.27 $\times$** for the Extended Kalman Filter.
5. **Modular Design Simplicity:** By maintaining identical function signatures, register conventions, and naming boundaries across milestones, the integration process required zero structural modifications to our existing C-language drivers or main filter loops. Transitioning the system from scalar execution to vector acceleration was reduced to a simple compile-time linking decision, demonstrating excellent software engineering practices.

9.1 Closing Remarks

In conclusion, this project successfully bridges the gap between high-level algorithms and low-level hardware acceleration. While algorithmic dependencies (such as the sequential steps in the Cholesky decomposition and CORDIC math) and software emulation parameters (such as QEMU’s dynamic binary translation tax) established concrete performance boundaries, the dramatic compression of absolute instruction counts proves the structural excellence of the vector design. Moving this implementation from a software simulator onto physical RISC-V hardware with a dedicated vector processing unit would immediately unlock the full parallel velocity of our code. Ultimately, Milestone 4 delivers a robust, highly optimized, and mathematically flawless ecosystem, validating the exceptional power of the RISC-V Vector extension for real-time signal processing and complex state estimation workloads.

9.2 Code Availability and Repository Link

The complete open-source code for this project, including all vectorised assembly files, verification datasets, and the compilation setup, can be accessed directly on GitHub at:

https://github.com/hamzi-codes/CAAL_S26_Project_KalmanFilter_Team-Thunder/tree/milestone-4